

CAPSTONE PROJECT REPORT

Automate a Web Application Using Selenium WebDriver with Python

Project Title

E-Commerce Web Application Automation Testing

Student Name	Arghadeep Saha
Programme	B.Tech in Computer Science and Engineering (AI & ML)
Batch	2023–2027
Subject	Python Automation
Topic	Web Application Using Selenium WebDriver with Python

1. Module Name and Experiment Title

Module Name: Capstone Project – Web Application Automation

Experiment Title: Automate a Web Application Using Selenium WebDriver with Python

Project Theme: Automated testing of a demo e-commerce workflow covering browser launch, account/login navigation, product search, product selection, cart operations, quantity update, verification, screenshots, and test reporting.

2. Problem Statement

Manual testing of an e-commerce web application requires a tester to repeatedly perform the same sequence of browser actions and verify the resulting page state. Repetitive manual execution can be time-consuming and can make regression testing difficult. The problem addressed in this experiment is to automate a representative e-commerce shopping workflow using Selenium WebDriver with Python and PyTest, while validating important outcomes.

3. Objective

- To automate a web application using Selenium WebDriver and Python.
- To practice locating web elements using ID, NAME, LINK_TEXT, CSS Selector, and related Selenium locator strategies.
- To automate product search and cart operations on a demo e-commerce website.
- To update the quantity of a selected product and verify the final cart state using assertions.
- To capture screenshots at important stages of execution.
- To execute the automation through PyTest and generate an HTML execution report.

4. Tools, Software, and Concepts Used

Tool / Concept	Usage
Programming Language	Python
Automation Framework	Selenium WebDriver
Testing Framework	PyTest
Browser	Google Chrome
Reporting	pytest-html
Website	TutorialsNinja Demo
Editor	Visual Studio Code
Key Concepts	Web locators, browser automation, assertions, screenshots, test execution and reporting

5. Implementation Steps

Step 1 – Launch Browser: Create a Chrome WebDriver instance and maximize the browser window.

Step 2 – Open Website: Navigate to <https://tutorialsninja.com/demo/>.

Step 3 – Login Navigation: Open My Account → Login, enter the test email and password, and click the login submission button.

Step 4 – Search Product: Locate the search field by NAME, enter “MacBook”, and click the search button using a CSS Selector.

Step 5 – Open Product: Locate the MacBook product using LINK_TEXT and open its product page.

Step 6 – Add to Cart: Locate the Add to Cart button using ID and click it.

Step 7 – Open Cart: Open the cart and select View Cart.

Step 8 – Update Quantity: Locate the quantity input using CSS Selector, replace the quantity with 2, and click Update.

Step 9 – Verify Cart: Assert that “MacBook” is present in the page source and assert that the quantity value is 2.

Step 10 – Capture and Close: Save the final screenshot, print execution messages, and close the WebDriver session.

6. Source Code

The following is the complete test script used for the capstone demonstration. It performs the complete workflow within one pytest test function.

```
from selenium import webdriver

from selenium.webdriver.common.by import By

import time

def test_ecommerce_purchase():

    driver = webdriver.Chrome()

    driver.maximize_window()

    driver.get("https://tutorialsninja.com/demo/")

    time.sleep(2)

    driver.find_element(By.LINK_TEXT, "My Account").click()

    time.sleep(1)

    driver.find_element(By.LINK_TEXT, "Login").click()

    time.sleep(2)

    driver.find_element(By.ID, "input-email").send_keys("test@example.com")
```

```
driver.find_element(By.ID, "input-password").send_keys("test123")

driver.find_element(

    By.CSS_SELECTOR, "input[type='submit']"

).click()

time.sleep(2)

search = driver.find_element(By.NAME, "search")

search.send_keys("MacBook")

driver.find_element(

    By.CSS_SELECTOR, "button.btn.btn-default.btn-lg"

).click()

time.sleep(3)

driver.save_screenshot(

    "screenshots/01_product_search.png"

)

driver.find_element(By.LINK_TEXT, "MacBook").click()

time.sleep(2)

driver.find_element(By.ID, "button-cart").click()

time.sleep(3)

driver.save_screenshot(

    "screenshots/02_product_added.png"

)

driver.find_element(By.ID, "cart-total").click()

time.sleep(1)

driver.find_element(By.LINK_TEXT, "View Cart").click()

time.sleep(3)

quantity = driver.find_element(

    By.CSS_SELECTOR, "input[name^='quantity']"

)
```

```
quantity.clear()

quantity.send_keys("2")


driver.find_element(
    By.CSS_SELECTOR,
    "button[data-original-title='Update']"
).click()

time.sleep(3)

assert "MacBook" in driver.page_source

quantity = driver.find_element(
    By.CSS_SELECTOR, "input[name^='quantity']"
)

assert quantity.get_attribute("value") == "2"

driver.save_screenshot(
    "screenshots/03_cart_verified.png"
)

print("Product search successful")

print("Product added to cart")

print("Quantity updated to 2")

print("Cart verification successful")

driver.quit()
```

7. Output and Clearly Labelled Screenshots

The automation saves three screenshots during execution. The actual screenshot files can be inserted in this section after running the test on the submission computer.

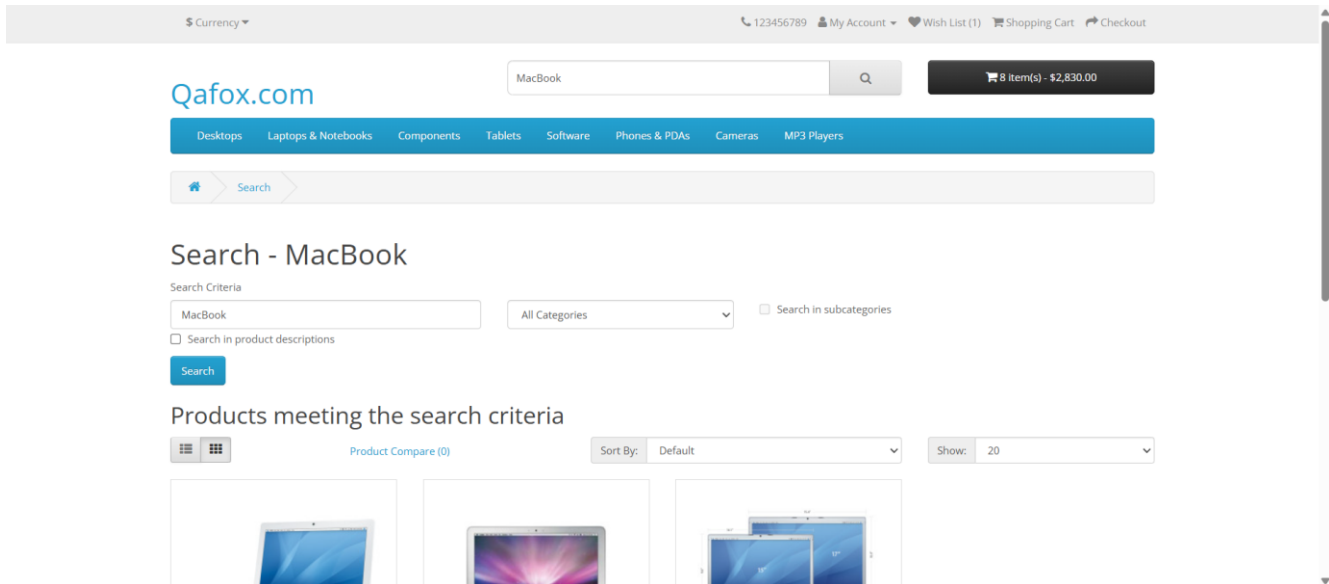


Figure 1 – Product Search

Screenshot captured after searching for the MacBook product.

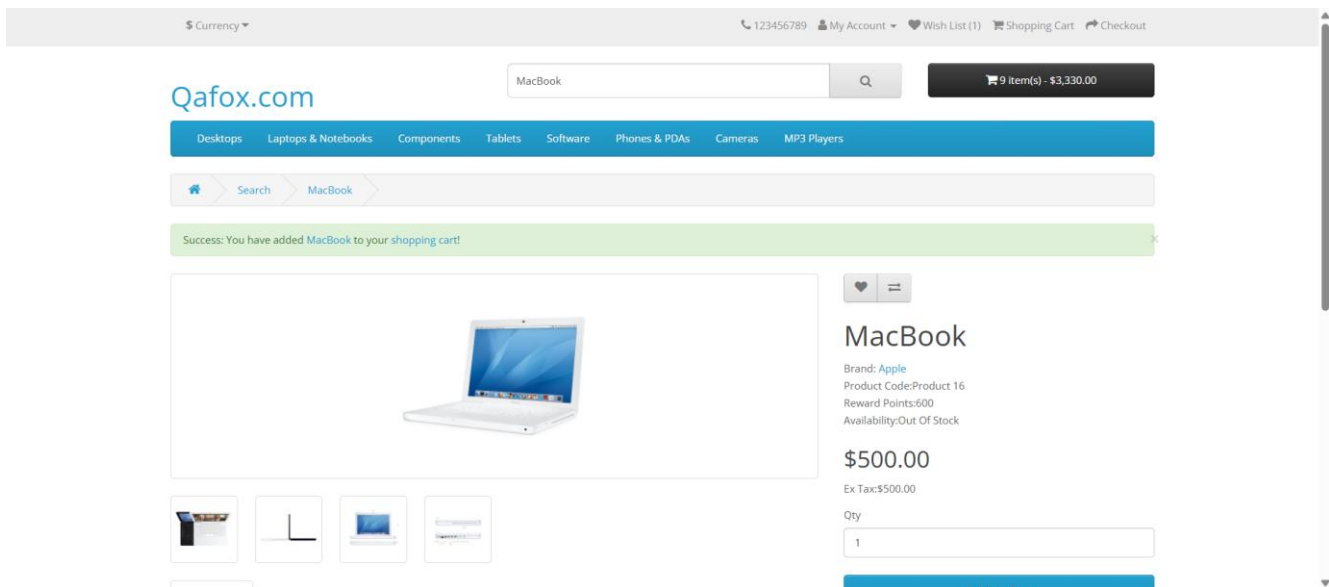


Figure 2 – Product Added

Screenshot captured after adding the MacBook to the shopping cart.

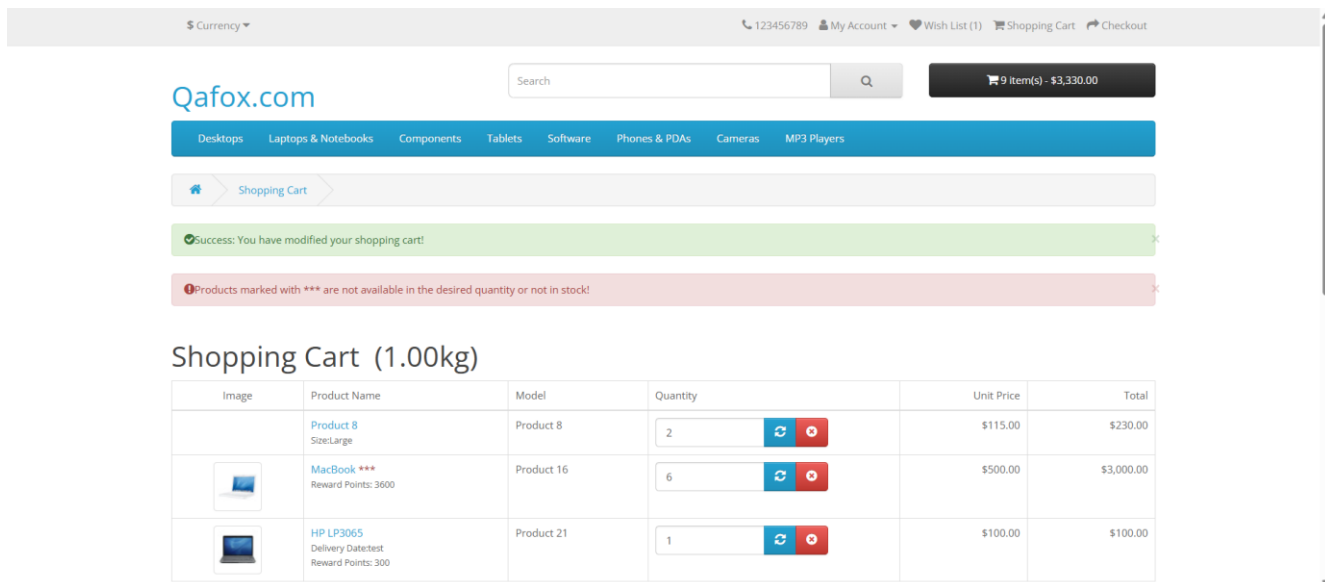


Figure 3 – Cart Verification

Screenshot captured after changing the quantity to 2 and verifying the cart.

8. Test Execution and Report

The test is executed from the project directory using the following PyTest command:

```
python -m pytest -v --html=reports/report.html --self-contained-html
```

The expected successful execution result is one passed test case. The generated HTML report is stored in the reports folder as report.html.

9. Result and Observation

Result: The complete Selenium automation workflow executes as a single PyTest test case. The script performs the defined browser interactions, captures screenshots, and uses assertions to verify that the MacBook is present in the cart and that its quantity is updated to 2.

Observation: The experiment demonstrates that Selenium can automate repetitive browser actions and that PyTest assertions can be used to validate expected web application states. The generated screenshots provide visual evidence of important execution stages, while the HTML report provides a record of test execution.

10. Conclusion

This capstone experiment successfully demonstrates web application automation using Selenium WebDriver with Python and PyTest. The project covers browser control, web element identification, product search, cart interaction, quantity modification, assertions, screenshot capture, and HTML reporting. The work provides practical experience in structuring and executing an automated web test for a representative e-commerce workflow.